



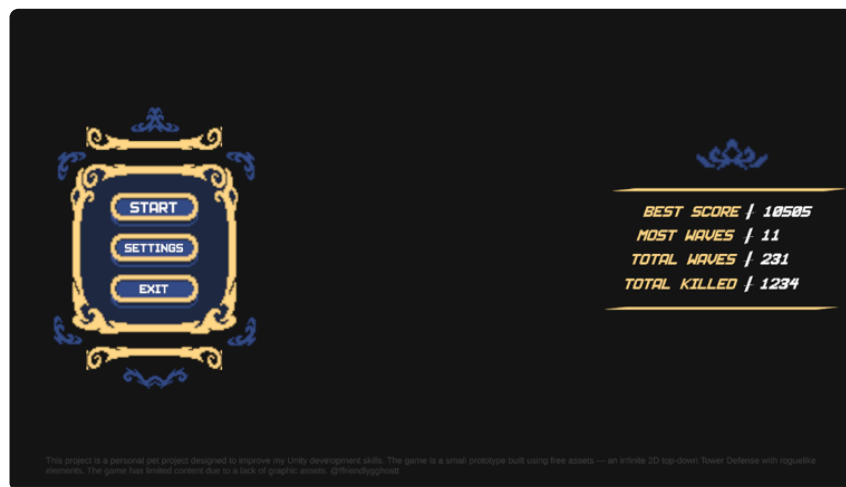
Save System: SaveManager, SaveData, GameManager, Records Panel, SettingsData

Save?

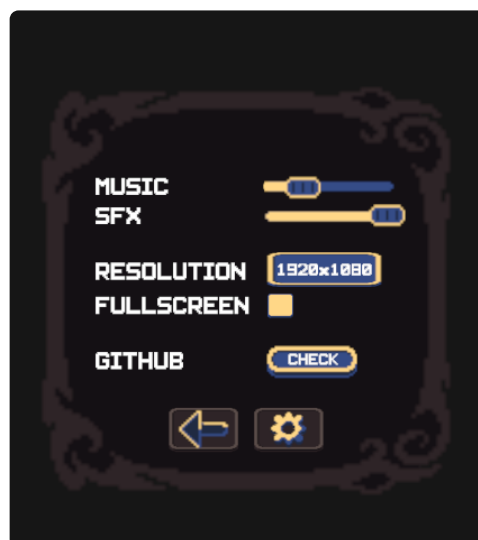
Практически все игры требуют наличие системы сохранений. Самая минимальная - сохранение рекорда игрока. Более глобальная - всех систем прогрессии игрока (открытые уровни, пройденная часть сюжета, найденные предметы, расположение их в инвентаре, открытые навыки и прочее).

В моём случае достаточно будет системы сохранения, которая будет держать рекорд по очкам, волнам, общему количеству волн и убийств. Вместе с тем, хорошо бы сохранять настройки игры: громкость музыки, звуков, разрешение, режим окна. Во многих демо не запариваются с сохранением настроек, поэтому при перезаходе их приходится выбирать снова. Я в качестве опыта решил реализовать подобное.

Раз у меня планируется наличие артефактов, которые будут выдаваться игроку после прохождения этажа, можно было бы сделать справочник таких артефактов. Тогда бы понадобилось сохранять информацию об открытых и закрытых предметах. Но я считаю, что можно обойтись и без этого.



Главное меню (незаконченно).



SettingsPanel.

SaveData.

Чтобы хранить всё одним классом, сделаем SaveData, который, в свою очередь, будет содержать другие классы-хранилища.

PlayerStatsData - под рекорды. SettingsData - под настройки.

```
1 using UnityEngine;
2
3 [System.Serializable]
4 public class SaveData
5 {
6     public PlayerStatsData stats = new PlayerStatsData();
7     public SettingsData settings = new SettingsData();
8 }
9
10
11
12 [System.Serializable]
13 public class PlayerStatsData
14 {
15     public int bestScore;
16     public int mostWaves;
17     public int totalWaves;
18     public int totalKilled;
19 }
20
21
22
23 [System.Serializable]
24 public class SettingsData
25 {
26     public float musicVolume = 1f;
27     public float sfxVolume = 1f;
28     public bool isFullScreen = true;
29     public int resolutionIndex = 0;
30 }
```

Records Panel и анимация бегущих чисел.

В скрипте панели рекордов, при включении объекта, мы получаем данные из менеджера и закидываем их на интерфейс.

```
1 using TMPro;
2 using UnityEngine;
3 using UnityEngine.UI;
4
5 public class RecordsPanel : MonoBehaviour
6 {
7     [SerializeField] public TextMeshProUGUI bestScoreText;
8     [SerializeField] public TextMeshProUGUI mostWavesText;
9     [SerializeField] public TextMeshProUGUI totalWavesText;
10    [SerializeField] public TextMeshProUGUI totalKilledText;
11
12    private void OnEnable()
13    {
14        var stats = SaveManager.Instance.Data.stats;
15
16        bestScoreText.text = stats.bestScore.ToString();
17        mostWavesText.text = stats.mostWaves.ToString();
18        totalWavesText.text = stats.totalWaves.ToString();
19        totalKilledText.text = stats.totalKilled.ToString();
20    }
21 }
```

Статичный текст мне показался скучным и я думал что бы добавить... Мне пришло на ум сделать простенькую анимацию с переборкой цифр. Каждые пару секунд идёт переборка цифр в случайном порядке, а потом останавливается на нужном значении.

У меня будет бесконечный цикл, который будет крутить корутину, а после запускать задержку перед повторной анимацией.

```
1 using System.Collections;
```

```

2 using TMPro;
3 using UnityEngine;
4 using System.Text.RegularExpressions;
5 using System.Collections;
6
7 public class RecordsAnimator : MonoBehaviour
8 {
9     [Header("Fields")]
10    public TMP_Text[] recordFields;
11
12    [Header("Settings")]
13    public float initialDelay = 2.5f;
14    public float animationDuration = 2f;
15    public float repeatDelay = 7f;
16    public float updateInterval = 0.05f;
17
18    void Start()
19    {
20        StartCoroutine(AnimateRecordsLoop());
21    }
22
23    IEnumerator AnimateRecordsLoop()
24    {
25        yield return new WaitForSeconds(initialDelay);
26
27        while (true)
28        {
29            yield return StartCoroutine(AnimateRecords());
30            yield return new WaitForSeconds(repeatDelay);
31        }
32    }
33
34    IEnumerator AnimateRecords()
35    {
36        float timer = 0f;
37
38        int[] targetNumbers = new int[recordFields.Length];
39
40        // Сначала мы собираем все данные себе в массив.
41
42        for (int i = 0; i < recordFields.Length; i++)
43        {
44            string text = recordFields[i].text;
45            int num = 0;
46
47            Match match = Regex.Match(text, @"\d+");
48
49            /*
50             Попробуем из текста получить число. К слову, текст "MaxWave" -
51             у меня идёт отдельным объектом, и Regex можно было не использовать, но
52             @ - технический символ, чтобы работал обратный слеш.
53             \d - любая цифра (0-9).
54             + - одна или более подряд.
55             */
56
57            if (match.Success)
58            {
59                num = int.Parse(match.Value);
60            }
61
62            targetNumbers[i] = num;
63        }
64
65        /*
66         То есть в определенный момент, если бы текст был цельным:
67         Максимум волн: 237, то метод пробежался бы по массиву символов,
68         дошёл до 2, и потом собирал бы цифры, пока снова не начались бы другие
69         В итоге мы получим 237. У меня цифры идут отдельным текстом, но как за
70         предусмотреть можно. Далее мы парсим строку в int тип.
71         */
72
73        while (timer < animationDuration)
74        {
75            timer += updateInterval;
76
77            for (int i = 0; i < recordFields.Length; i++)
78            {
79                int digits = targetNumbers[i].ToString().Length;
80                int randomValue;
81
82                switch (digits)
83                {
84                    case 1: randomValue = Random.Range(0, 10); break;

```

```

84         case 2: randomValue = Random.Range(10, 100); break
85         case 3: randomValue = Random.Range(100, 1000); bre
86         case 4: randomValue = Random.Range(1000, 10000); b
87         case 5: randomValue = Random.Range(10000, 100000);
88         default: randomValue = targetNumbers[i]; break;
89     }
90
91     recordFields[i].text = randomValue.ToString();
92 }
93
94     yield return new WaitForSeconds(updateInterval);
95 }
96
97 /*
98     Запускаем сам цикл, длинной в некоторое время.
99     Через for присваиваем в каждое поле случайное число.
100     Но не абы какое, а равное по ширине тому, что изначально у нас находит
101     Если волн 237 - значит анимация будет от 100 - 999.
102     После чего мы просто выводим число на экран. И ожидаем некоторую задерж
103     Она кажется совсем маленькой = 0.05 секунды, однако без неё анимации б
104     While бы просто срабатывал ультра быстро и выводил бы мгновенно прошлы
105     а так мы наблюдаем анимацию безущих цифр.
106 */
107
108     for (int i = 0; i < recordFields.Length; i++)
109     {
110         recordFields[i].text = targetNumbers[i].ToString();
111     }
112 // В конце просто возвращаем на экран реальные значения.
113 }
114 }

```

SaveManager.

```

1  using System.IO;
2  using UnityEngine;
3
4  public class SaveManager : Manager<SaveManager>
5  {
6      private string SavePath => Path.Combine(Application.persistentDataP
7      "save.json");
8      // Получаем путь к папке под сохранение, в нашем случае это:
9      // C:\Users\ИмяПользователя\AppData\LocalLow\НазваниеКомпании\НазваниеИ
10
11
12  /*
13      Интересный факт (запишу для себя в том числе): если я захочу закинуть
14      сайт itch.io, который позволяет играть в браузере, то что будет с сохра
15      Для браузера мне нужно будет скомпилировать WebGL. Тогда она сможет зап
16      в браузере, а также Unity сам перенаправил запрос к папке
17      Application.persistentDataPath во внутреннюю базу данных браузера,
18      которая называется IndexedDB (аналог куки).
19      Данные будут сохраняться не где-то в папке под игру, а в кэше браузера.
20      Нейронка также посоветовала мне добавить:
21
22      #if UNITY_WEBGL && !UNITY_EDITOR
23          PlayerPrefs.Save();
24      #endif
25
26      В местах, где я буду делать запись в JSON, чтобы в случае WebGL билд
27      такой вызов принудительно заставлял браузер синхронизировать память и ж
28      зафиксировать все изменения в файле сразу же. Даже если я не использую
29      PlayerPrefs.
30  */
31      public SaveData Data { get; private set; }
32
33      protected override void Awake()
34      {
35          base.Awake();
36          Load();
37      }
38
39  /*
40      Сразу же в авейк мы подгружаем данные. Если данных нету
41      (как при первом запуске) → мы создаём пустой SaveData и сохраняем его в
42      Если данные есть, то считываем Json строку и расфасовываем в Data
43      (благо данные простые, а JsonUtility уже давно создан опытными ребятами
44      поэтому всё само расфасовывается).
45  */
46

```

```

47     public void Save()
48     {
49         string json = JsonUtility.ToJson(Data, true);
50         File.WriteAllText(SavePath, json);
51         #if UNITY_WEBGL && !UNITY_EDITOR
52             PlayerPrefs.Save();
53         #endif
54     }
55
56     // Когда нам нужно сохранить, то мы просто берём текущую Data и сохраняем
57     // Соответственно, с текущей Data работать должен кто-то извне...
58
59     public void Load()
60     {
61         if (File.Exists(SavePath))
62         {
63             string json = File.ReadAllText(SavePath);
64             Data = JsonUtility.FromJson<SaveData>(json);
65         }
66         else
67         {
68             Data = new SaveData();
69             Save();
70         }
71     }
72
73     public void ResetData()
74     {
75         Data = new SaveData();
76         Save();
77     }
78 }

```

GameManager.

i Для управления состояниями игры и контролем многих менеджеров используется GameFlowManager. GameManager же содержит более поверхностную информацию, которая как раз нужна для сохранения рекордов.

Публичные методы вызываются из GameFlowManager'a и других. К примеру, если враг именно убит, а не просто дошёл до конца базы, то у него вызовется: GameManager.Instance.AddScore(Exp); Если волна закончилась, то спавнер вызовет: GameManager.Instance.WaveCleared();

Все характеристики хранятся один забег. Как только игрок проигрывает, вызывается EndRun(); Он получает данные из менеджера сохранений, сумирует общие характеристики, а другие сравнивает. После чего мы вызываем сохранение и сбрасываем текущие поля.

GetRunStats() - в текущем менеджере используется для того, чтобы показать на экране смерти статусы текущего забега. Мы просто упаковываем поля в структуру и выкидываем её в DeathScreen.

```

1  using UnityEngine;
2
3  public struct RunStats
4  {
5      public int kills;
6      public int score;
7      public int waves;
8      public int floor;
9      public float runTime;
10 }
11
12 public class GameManager : Manager<GameManager>
13 {
14     private int killsThisRun;
15     private int scoreThisRun;
16     private int wavesCleared;
17     private int floorThisRun;
18     public int FloorThisRun => floorThisRun;
19
20     private float runTime;
21     private bool isRunning = false;
22 }

```

```

23     public void AddKill()
24     {
25         killsThisRun++;
26     }
27
28     public void AddScore(int points)
29     {
30         scoreThisRun += points;
31     }
32
33     public void WaveCleared()
34     {
35         wavesCleared++;
36     }
37
38     public void AddFloor()
39     {
40         floorThisRun++;
41     }
42
43     public void EndRun()
44     {
45         var stats = SaveManager.Instance.Data.stats;
46         stats.totalKilled += killsThisRun;
47         stats.totalWaves += wavesCleared;
48
49         if (scoreThisRun > stats.bestScore)
50         {
51             stats.bestScore = scoreThisRun;
52         }
53
54         if (wavesCleared > stats.mostWaves)
55         {
56             stats.mostWaves = wavesCleared;
57         }
58
59         SaveManager.Instance.Save();
60         Reset();
61     }
62
63     private void Update()
64     {
65         if (!isRunning) return;
66         runTime += Time.deltaTime;
67     }
68
69     public void StartTimer()
70     {
71         runTime = 0;
72         isRunning = true;
73     }
74
75     public void EndTimer()
76     {
77         isRunning = false;
78     }
79
80
81     public RunStats GetRunStats()
82     {
83         return new RunStats
84         {
85             kills = killsThisRun,
86             score = scoreThisRun,
87             waves = wavesCleared,
88             floor = floorThisRun,
89             runTime = runTime
90         };
91     }
92
93     private void Reset()
94     {
95         killsThisRun = 0;
96         scoreThisRun = 0;
97         wavesCleared = 0;
98         floorThisRun = 0;
99     }
100
101     public void StartGame()
102     {
103         StartTimer();
104     }

```



Настройки игры меняются в реальном времени, а сохраняются при закрытии меню настроек. Подгружаются настройки один раз в специальный `SettingsService`, при заходе в игру. Подробнее на странице `Settings Systems`.